

Niezawodność systemów informatycznych

Dokumentacja systemu testowanego (SUT)
Moduł BankSystem

Semestr 7, rok akademicki 2026/2027

1 Do czego służy ten dokument

To skrócona dokumentacja API systemu testowanego (SUT: *System Under Test*), na którym pracujecie przez cały semestr. Znajdziecie tu opis wszystkich klas, metod, parametrów i wyjątków modułu **BankSystem**, wraz z przykładami użycia. Dokument nie opisuje, które zachowania systemu są poprawne a które nie: to wy macie to ustalić w ramach projektu, testując, czytając kod i analizując wyniki. Traktujcie ten plik jak dokumentację biblioteki, z którą pracujecie, a nie jak podręcznik do zajęć.

Jak korzystać z tego dokumentu

Nie musicie czytać wszystkiego po kolei. Najwygodniej jest otworzyć ten dokument razem z plikiem `zesp1_X.py` i sprawdzać opis metody w momencie, gdy jej używacie albo testujecie.

To nie jest wzorzec poprawności

Dokumentacja opisuje *interfejs* systemu: jakie metody istnieją, jakie parametry przyjmują i co deklaratorywnie powinny zwracać. Nie jest to zapewnienie, że każdy wariant SUT rzeczywiście działa tak, jak tu opisano we wszystkich przypadkach. Weryfikacja tego jest częścią waszego projektu.

2 Ogólna budowa systemu

System symuluje uproszczony system bankowy. Składa się z trzech elementów:

- **BankSystem**: główna klasa systemu, przechowuje wszystkie konta i udostępnia operacje bankowe (wpłaty, wypłaty, przelewy, odsetki, historię, wyciągi).
- **Konto**: reprezentuje pojedyncze konto bankowe (numer, właściciel, saldo, historia operacji, status).
- **ZewnetrznyProcesor**: symulacja zewnętrznego systemu rozliczeniowego, przez który realizowane są przelewy wychodzące poza system (np. do innego banku). Może być zawodny: patrz sekcja 7.

Dodatkowo moduł definiuje trzy wyjątki (**KontoNieIstnieje**, **KontoZablokowane**, **ProcesorNiedostepny**) oraz jedną stałą konfiguracyjną:

```
1 LIMIT_DZIENNY_WYPLAT = Decimal("5000")
```

określającą maksymalną sumę wypłat z jednego konta w ciągu jednego dnia kalendarzowego.

3 Format numeru konta

Numer konta to **ciąg znaków** o długości 10, złożony wyłącznie z cyfr. Dziesiąta cyfra jest cyfrą kontrolną, wyliczaną w następujący sposób:

Algorytm cyfry kontrolnej

Niech $c_1, c_2, \dots, c_9$ będą pierwszymi dziewięcioma cyframi numeru konta, a c_{10} dziesiątą (ostatnią) cyfrą. Numer jest poprawny, gdy:

$$c_{10} = \left(\sum_{i=1}^9 c_i \right) \bmod 10$$

Przykład: dla numeru 1234567895 suma pierwszych dziewięciu cyfr to $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$, a $45 \bmod 10 = 5$, co zgadza się z ostatnią cyfrą. Numer jest poprawny.

Szybkie generowanie poprawnych numerów

Jeśli potrzebujecie wielu poprawnych numerów testowych, wygodniej jest napisać krótką funkcję pomocniczą, która dla dowolnych dziewięciu cyfr dolicza poprawną cyfrę kontrolną, niż wymyślać numery metodą prób i błędów.

Próba utworzenia konta z numerem, który nie spełnia tego wzoru (albo nie jest ciągiem dziesięciu cyfr), zakończy się zgłoszeniem wyjątku `ValueError`.

4 Wyjątki

Wyjątek	Kiedy jest zgłaszany
<code>KontoNieIstnieje</code>	Operacja odwołuje się do numeru konta, który nie został wcześniej utworzony metodą <code>utworz_konto</code> .
<code>KontoZablokowane</code>	Próba wypłaty, przelewu lub przelewu zewnętrznego z konta, które jest w stanie "zablokowane".
<code>ProcesorNiedostepny</code>	Zewnętrzny procesor płatności (sekcja 7) odmówił obsłużenia przelewu.
<code>ValueError</code>	Nieprawidłowe dane wejściowe: zły numer konta, ujemna lub zerowa kwota, niewystarczające środki, przekroczony limit.
<code>TypeError</code>	Kwota operacji nie jest liczbą (<code>int</code> , <code>float</code> lub <code>Decimal</code>).

5 Klasa Konto

Obiekt `Konto` reprezentuje jedno konto bankowe. W normalnym użyciu nie tworzy się go samodzielnie: powstaje automatycznie przy wywołaniu `BankSystem.utworz_konto`.

Atrybuty

Atrybut	Opis
<code>numer</code>	Numer konta (string, 10 cyfr).
<code>wlasciciel</code>	Nazwa lub imię i nazwisko właściciela konta (string).
<code>saldo</code>	Aktualne saldo konta, typu <code>Decimal</code> .
<code>historia</code>	Lista wpisów historii operacji, każdy jako trójka (<code>data</code> , <code>typ</code> , <code>kwota</code>).

Atrybut	Opis
<code>status</code>	"aktywne" albo "zablokowane".

Decimal, nie float

Saldo i kwoty operacji są przechowywane jako `decimal.Decimal`, nie jako `float`. To standardowa praktyka w systemach finansowych: `float` wprowadza błędy zaokrąglenia przy operacjach na pieniądzach. Jeśli piszecie własne testy porównujące kwoty, pamiętajcie o tym przy konwersjach typów.

6 Klasa `BankSystem`

Główny punkt wejścia do systemu. Jeden obiekt `BankSystem` przechowuje wszystkie konta w słowniku `self.konta` (klucz: numer konta).

```
1 bank = BankSystem()
```

6.1 `utworz_konto(numer, wlasciciel, saldo_poczatkowe=0)`

Tworzy nowe konto i zwraca obiekt `Konto`.

- **numer**: string, 10 cyfr, musi spełniać wzór cyfry kontrolnej (sekcja 3).
- **wlasciciel**: string, dowolna nazwa właściciela.
- **saldo_poczatkowe**: liczba, saldo konta w momencie utworzenia (domyślnie 0).

Zgłasza `ValueError`, jeśli numer konta jest nieprawidłowy albo konto z tym numerem już istnieje.

```
1 konto = bank.utworz_konto("1234567895", "Jan Kowalski", saldo_poczatkowe=1000)
```

6.2 `zablokuj_konto(numer) / odblokuj_konto(numer)`

Zmieniają status konta na "zablokowane" albo "aktywne". Obie metody zgłaszają `KontoNieIstnieje`, jeśli podany numer konta nie istnieje w systemie.

6.3 `wplac(numer, kwota)`

Zasila konto o podaną kwotę. Zwraca nowe saldo konta.

- Zgłasza `TypeError`, jeśli `kwota` nie jest liczbą.
- Zgłasza `ValueError`, jeśli `kwota` jest mniejsza lub równa zero.
- Zgłasza `KontoNieIstnieje`, jeśli konto nie istnieje.

6.4 `wyplac(numer, kwota)`

Obciąża konto podaną kwotą. Zwraca nowe saldo konta.

- Zgłasza `KontoZablokowane`, jeśli konto jest zablokowane.
- Zgłasza `ValueError`, jeśli `kwota` jest niedodatnia, przekracza saldo albo przekracza dzienny limit wypłat (`LIMIT_DZIENNY_WYPLAT`, domyślnie 5000, liczony łącznie dla wypłat i przelewów wychodzących danego dnia kalendarzowego).
- Zgłasza `KontoNieIstnieje`, jeśli konto nie istnieje.

6.5 przelew(numer_z, numer_do, kwota)

Przenosi środki między dwoma kontami w systemie. Operacja jest zabezpieczona blokadą (`threading.Lock`) na potrzeby jednoczesnych wywołań z wielu wątków. Zwraca saldo konta źródłowego po operacji.

- Zgłasza `KontoZablokowane`, jeśli konto źródłowe jest zablokowane.
- Zgłasza `ValueError`, jeśli kwota jest niedodatnia, przekracza saldo konta źródłowego albo przekracza globalny limit systemowy (1 000 000).
- Zgłasza `KontoNieIstnieje`, jeśli jedno z kont nie istnieje.

Dlaczego przelew używa blokady, a wpłata i wypłata nie

Przelew modyfikuje dwa konta na raz: odczyt i zapis muszą wyglądać jak jedna niepodzielna operacja, inaczej dwa wątki mogą "wpleść się" w siebie w trakcie aktualizacji sald. To częsty temat na zajęciach z niezawodności: operacje, które dotyczą więcej niż jednego zasobu współdzielonego, wymagają szczególnej uwagi przy współbieżności.

6.6 nalicz_odsetki(numer, roczna_stopa_procentowa, miesiace)

Dolicza do salda odsetki kapitalizowane miesięcznie. Stopa roczna jest dzielona przez 12 i przez 100, a wynik zaokrąglany do dwóch miejsc po przecinku (`ROUND_HALF_UP`) po każdym miesiącu. Zwraca saldo konta po doliczeniu odsetek za wszystkie podane miesiące.

```
1 bank.nalicz_odsetki("1234567895", roczna_stopa_procentowa=6, miesiace=12)
```

6.7 historia_w_okresie(numer, od, do)

Zwraca listę wpisów historii konta (trójki (`data`, `typ`, `kwota`)) dla operacji wykonanych w podanym przedziale dat, włącznie z obiema granicami. Parametry `od` i `do` to obiekty `datetime.date`.

6.8 wyciag_miesieczny(numer, rok, miesiac)

Zwraca słownik podsumowujący operacje na koncie w danym miesiącu:

```
1 {"wpłaty": Decimal(...), "wyплаты": Decimal(...), "saldo_koncowe": Decimal(...)}
```

`wpłaty` obejmuje wpłaty, przelewy przychodzące i odsetki; `wyплаты` obejmuje wypłaty i przelewy wychodzące.

6.9 przelew_zewnetrzny(numer_z, numer_konta_docelowego_zewn, kwota, procesor=None)

Realizuje przelew na konto spoza systemu (np. do innego banku), korzystając z zewnętrznego procesora płatności. Opisana szczegółowo w sekcji 7.

7 Klasa ZewnetrznyProcesor

`ZewnetrznyProcesor` symuluje zewnętrzny system rozliczeniowy (np. izbę rozliczeniową międzybankową), przez który idą przelewy wychodzące poza system. W odróżnieniu od reszty SUT, ten element jest **celowo zawodny**: jego zadaniem jest dać wam coś, na czym można przećwiczyć mechanizmy odporności (retry, circuit breaker, redundancja), a nie coś, co trzeba "naprawić".

```
1 procesor = ZewnetrznyProcesor(prawdopodobienstwo_awarii=0.3, opoznienie_s=0.0, generator=None)
```

- **prawdopodobienstwo_awarii**: liczba z przedziału $[0, 1]$, prawdopodobieństwo, że dane wywołanie zakończy się błędem (domyślnie 0,3, czyli około 30% wywołań).

- **opoznienie__s**: opcjonalne opóźnienie (w sekundach) przed odpowiedzią procesora, symulujące czas sieciowy.
- **generator**: opcjonalny obiekt `random.Random`. Przydatny, gdy chcecie w testach kontrolować, które wywołania się powiedą, a które nie (ustawiając generator z konkretnym `seed`).

Metoda `wyslij_przelew(numer_konta_docelowego, kwota)`

Z prawdopodobieństwem `prawdopodobienstwo_awarii` zgłasza wyjątek `ProcesorNiedostepny`. W przeciwnym razie zwraca słownik:

```
1 {"status": "zaksiegowano", "numer_referencyjny": "REF000001"}
```

Metoda `przelew_zewnetrzny` nie ponawia prób sama

`BankSystem.przelew_zewnetrzny` wywołuje procesor tylko raz. Jeśli procesor zgłosi `ProcesorNiedostepny`, wyjątek propaguje się na zewnątrz metody bez żadnej automatycznej próby ponowienia. Zaprojektowanie mechanizmu odpornego na tę niedostępność (retry z odroczeniem, circuit breaker, ewentualnie redundantne ścieżki) jest częścią zadań projektowych: nie jest to coś, co ma zrobić za was sam SUT.

Jak testować bez losowości

Żeby wasz test był powtarzalny, wstrzyknijcie własny procesor z kontrolowanym zachowaniem, np.:

```
1 import random
2 procesor_testowy = ZewnetrznyProcesor(prawdopodobienstwo_awarii=1.0, generator=random.
    Random(1))
3 bank.przelew_zewnetrzny(numer_z, numer_docelowy, 100, procesor=procesor_testowy)
4 # ta linia zawsze zgloszy ProcesorNiedostepny
```

8 Przykład użycia end-to-end

```
1 from decimal import Decimal
2
3 bank = BankSystem()
4 bank.utworz_konto("1234567895", "Jan Kowalski", saldo_poczatkowe=1000)
5 bank.utworz_konto("9876543215", "Anna Nowak")
6
7 bank.wplac("1234567895", 500)
8 bank.wyplac("1234567895", 200)
9 bank.przelew("1234567895", "9876543215", 300)
10
11 print(bank.konta["1234567895"].saldo) # saldo Jana po operacjach
12 print(bank.konta["9876543215"].saldo) # saldo Anny po operacjach
13
14 bank.nalicz_odsetki("1234567895", roczna_stopa_procentowa=4, miesiace=3)
15
16 try:
17     bank.przelew_zewnetrzny("1234567895", "0000000000", 50)
18 except ProcesorNiedostepny:
19     print("Przelew zewnetrzny nie powiodl sie, trzeba sprobowac ponownie")
```

9 Dane historyczne do modeli wzrostu niezawodności

W pliku `usterki_historyczne.csv` znajdziecie syntetyczny zbiór danych z wcześniejszego, dłuższego cyklu testowania podobnego systemu. Każdy wiersz to jedna wykryta usterka:

Kolumna	Opis
<code>nr_usterki</code>	Numer porządkowy wykrytej usterki.
<code>czas_wykrycia_h</code>	Czas wykrycia usterki, w godzinach od początku testowania.
<code>modul</code>	Nazwa modułu, w którym wykryto usterkę.
<code>dotkliwosc</code>	Dotkliwość usterki: drobna , powazna albo krytyczna .

Ten zbiór danych nie jest powiązany z żadnym konkretnym wariantem SUT przypisanym waszemu zespołowi: służy wyłącznie jako większy zestaw punktów danych do dopasowania modeli wzrostu niezawodności (np. Jelinski-Moranda, Goel-Okumoto) na bloku zajęć poświęconym estymacji niezawodności.